

Scaling LLM Inference on Kubernetes

A Practical Guide with vLLM and Ray

Haowen Huang

Senior Developer Advocate
Amazon Web Services

Dongdong Yang

Senior Container Specialist SA
Amazon Web Services



Content

- 01 Overview of LLM Inference on EKS
- 02 Model Deployment with vLLM
- 03 Observing LLM Inference Workloads
- 04 Scaling LLM Inference with vLLM & Ray



Part 01

Overview of LLM Inference on EKS

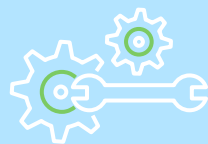


Why customers choose EKS for inference



Tooling ecosystem

Vibrant open-source system of tooling



Extensibility

Built-in Kubernetes native integrations with AWS ML infrastructure services



Cost optimization

Control of environment to deploy cost optimized generative AI applications



Configurability

Full control of your data and infrastructure configuration



Highly scalable

Full control of your data and infrastructure configuration



Run anywhere

Consistent deployments across clouds, on-premise, and at the edge



Operate less Ship more: Amazon EKS Auto Mode



Secure
by default



Easier and faster
to get started



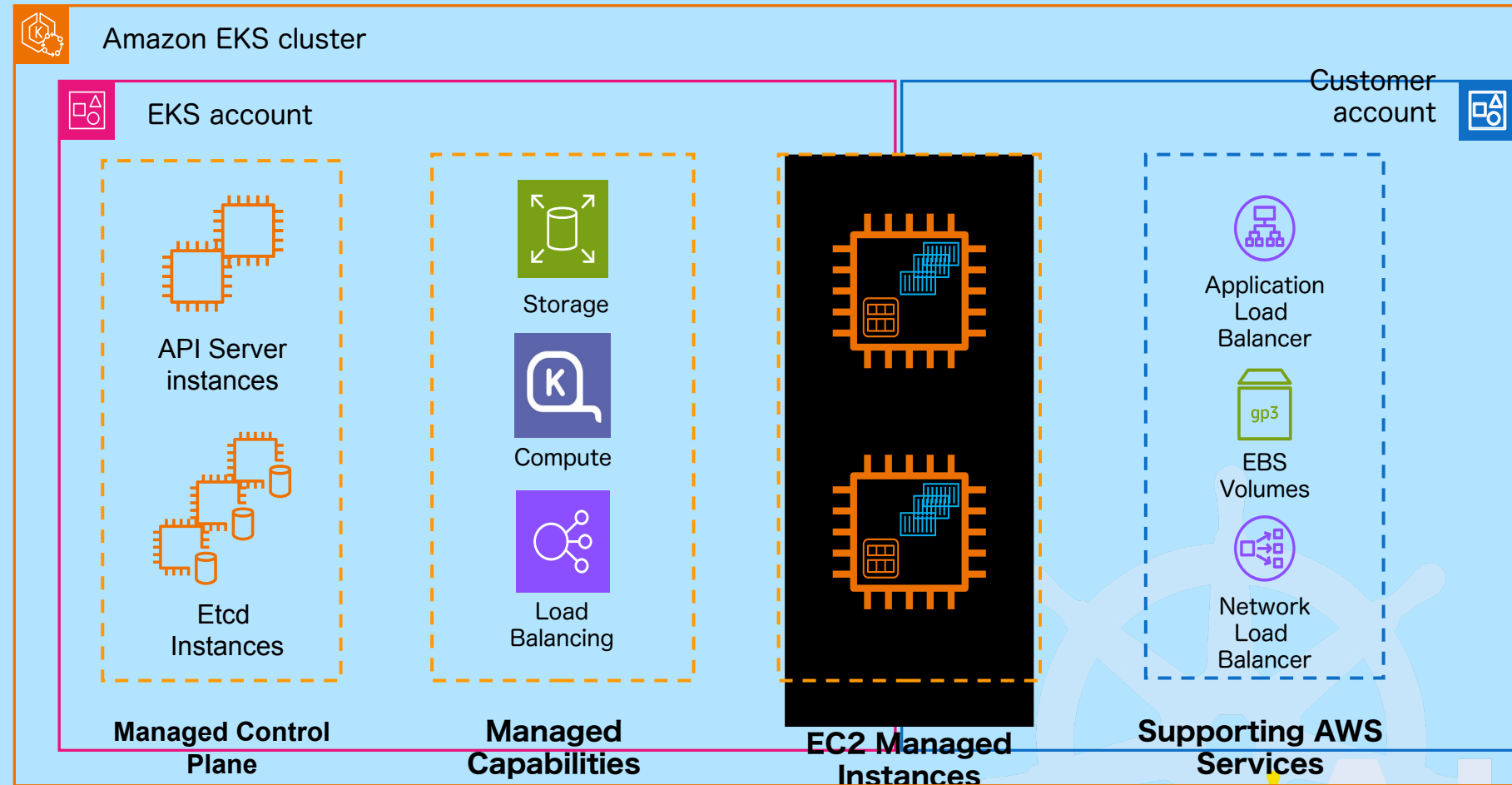
Access to nearly
all EC2 instances



Fully managed
core cluster
capabilities



Automatically
upgraded



Karpenter: GPU-Aware Node Provisioning

Open-source node lifecycle manager for Kubernetes

- Watches Pending Pods → provisions just-in-time capacity
- Direct EC2 Fleet API call (bypasses ASG/Node Groups)
- Automatic instance type selection for GPU workloads
- Built-in consolidation eliminates idle nodes

⚡ 2x Faster

0→100 nodes in ~2 min vs 4+ min (CA)

💰 Cost Optimized

Spot-first + auto consolidation

🎯 GPU-Aware

p5 / g5 / g6e auto-selection + fallback

CA + ASG

Multiple ASGs per type/AZ

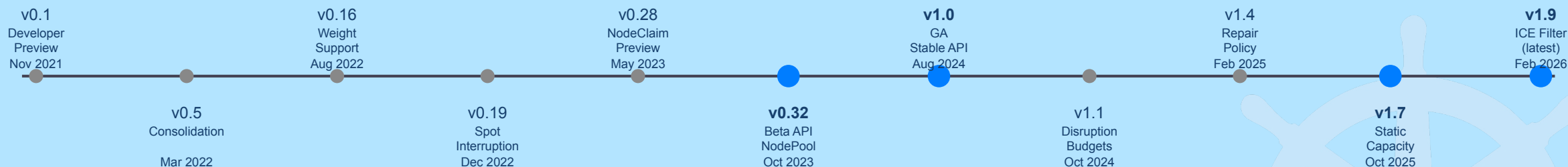
Slow scaling, no consolidation

Karpenter

Single NodePool, all types/AZs

JIT provisioning + auto right-size

Evolution



◆ v0.32 Provisioner → NodePool + EC2NodeClass

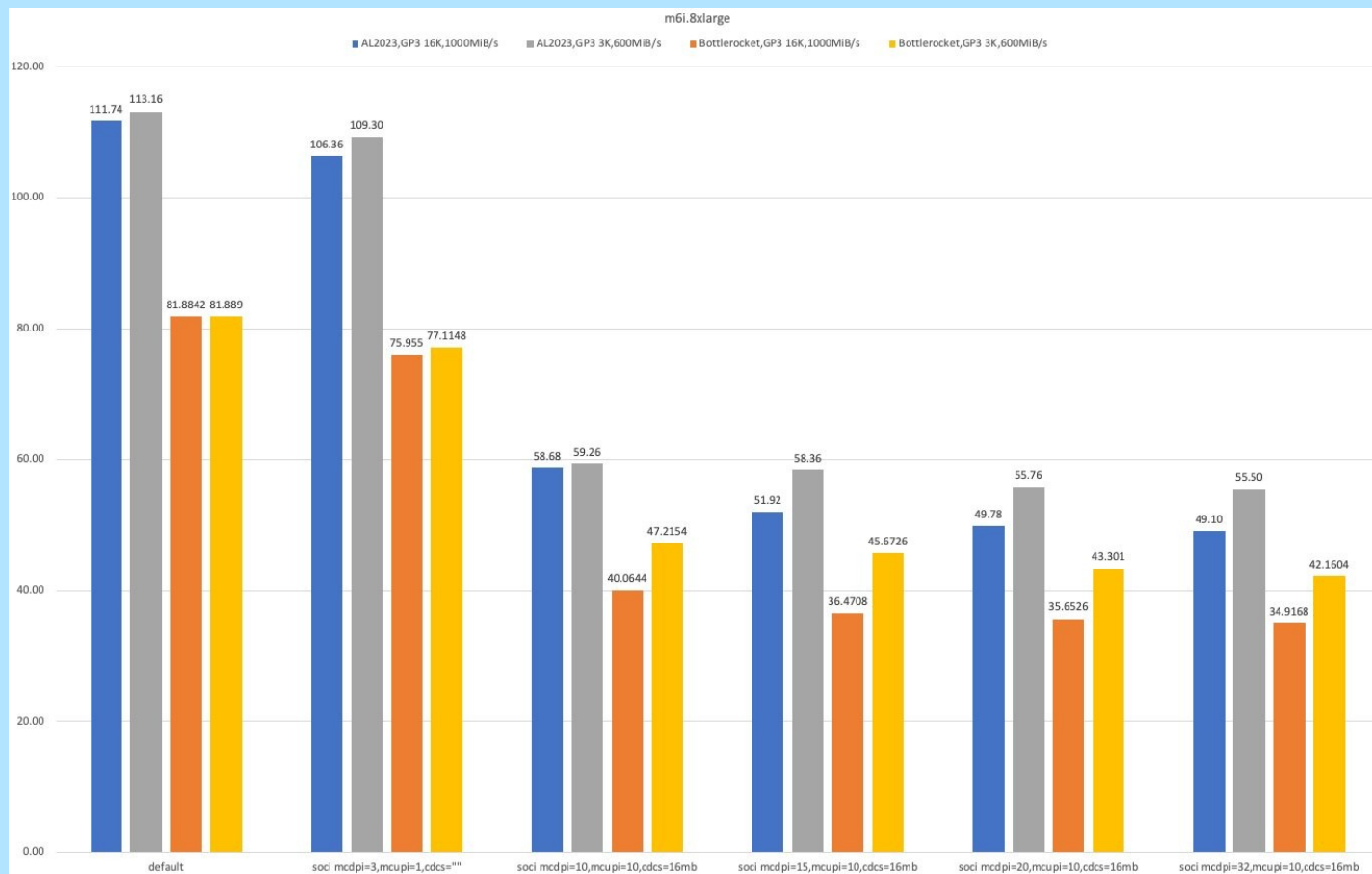
◆ v1.0 Stable API, production-ready

◆ v1.9 ICE filtering, g7et

karpenter.sh | github.com/aws/karpenter-provider-aws



Every layer, all at once: SOCI Parallel Pull Mode



SOCI Parallel Pull

- Multiple HTTP range requests per layer saturate network bandwidth
- Parallel decompression across layers utilizes all CPU cores
- Direct-to-disk writes keep memory usage constant
- No build-time image conversion required

Why not Lazy Loading?

- LLM images access nearly all data at startup
- Lazy loading defers but doesn't eliminate the full pull
- Runtime I/O jitter impacts inference latency

Impact

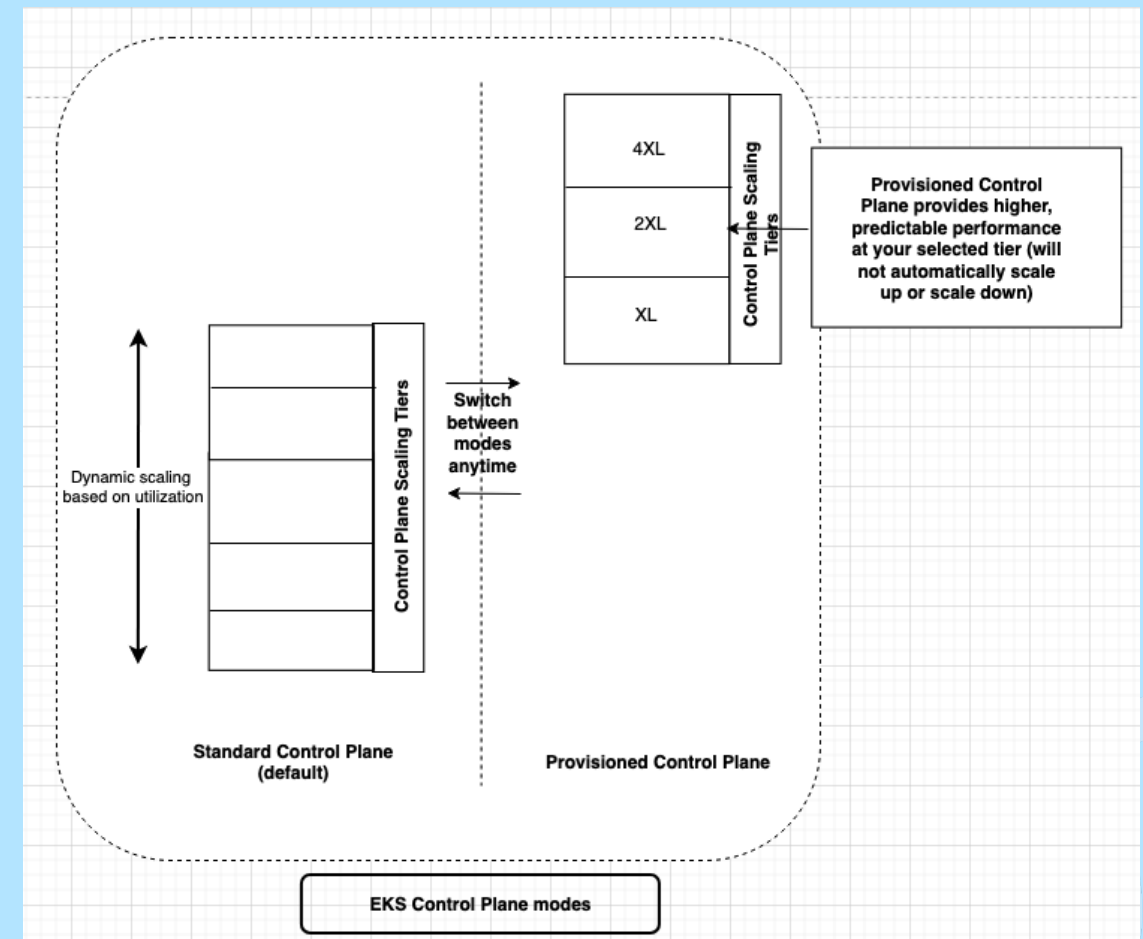
- 10GB+ LLM image pull: minutes → seconds
- EKS native: built into AL2023 & Bottlerocket AMIs
- Works with any OCI-compatible registry



Provisioned Control Plane in Amazon EKS

Ensure predictable and high performance for the most demanding workloads

- **API request concurrency:** the volume of requests processed by the Kubernetes API servers concurrently.
- **Pod scheduling rate:** the throughput at which the Kubernetes default scheduler assigns pods to nodes.
- **Cluster database size:** the storage space allocated to etcd, the database that holds the cluster state/metadata



Node health monitoring and auto repair

Comprehensive monitoring and remediation

Continuously monitors the health of node components including kubelet, CNI, disk, and CPU

Can automatically replace nodes when issues arise

Kubernetes native

Awareness of existing Kubernetes disruption controls such as Pod disruption budgets

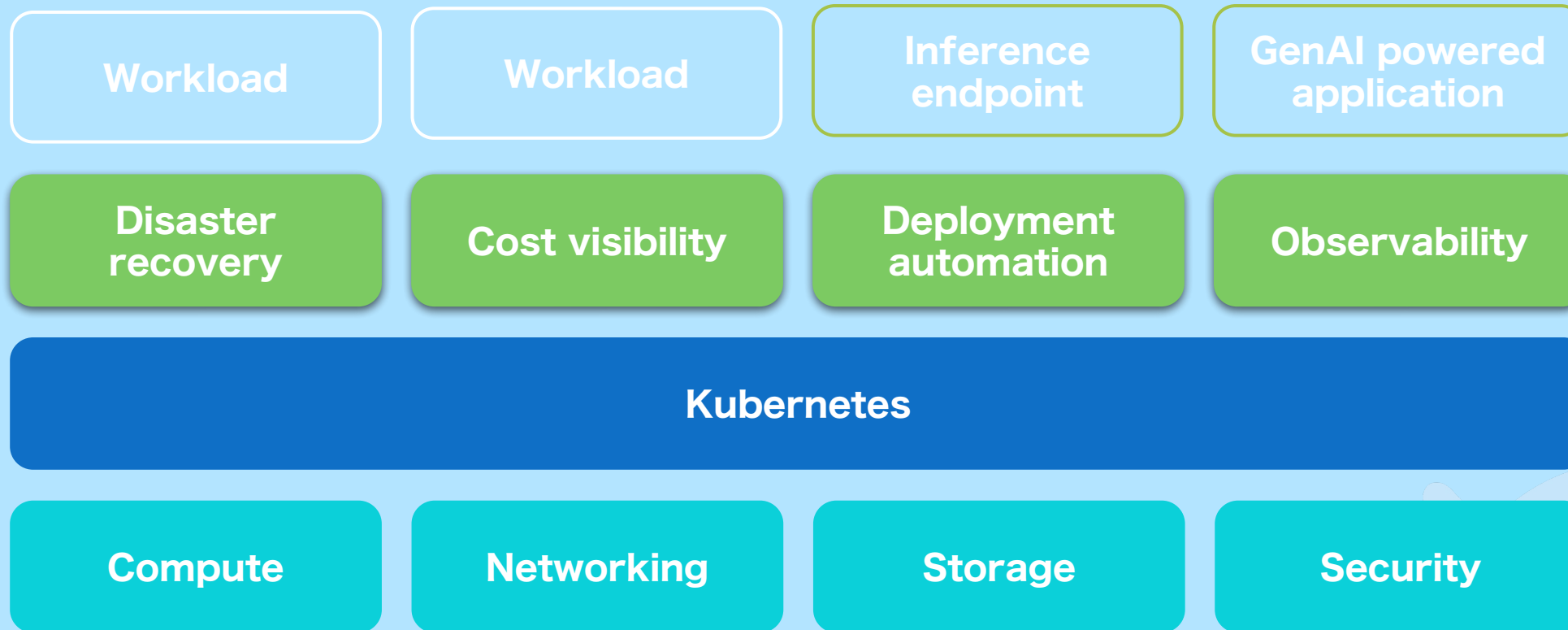
Enable with one click in EKS

GPU optimized

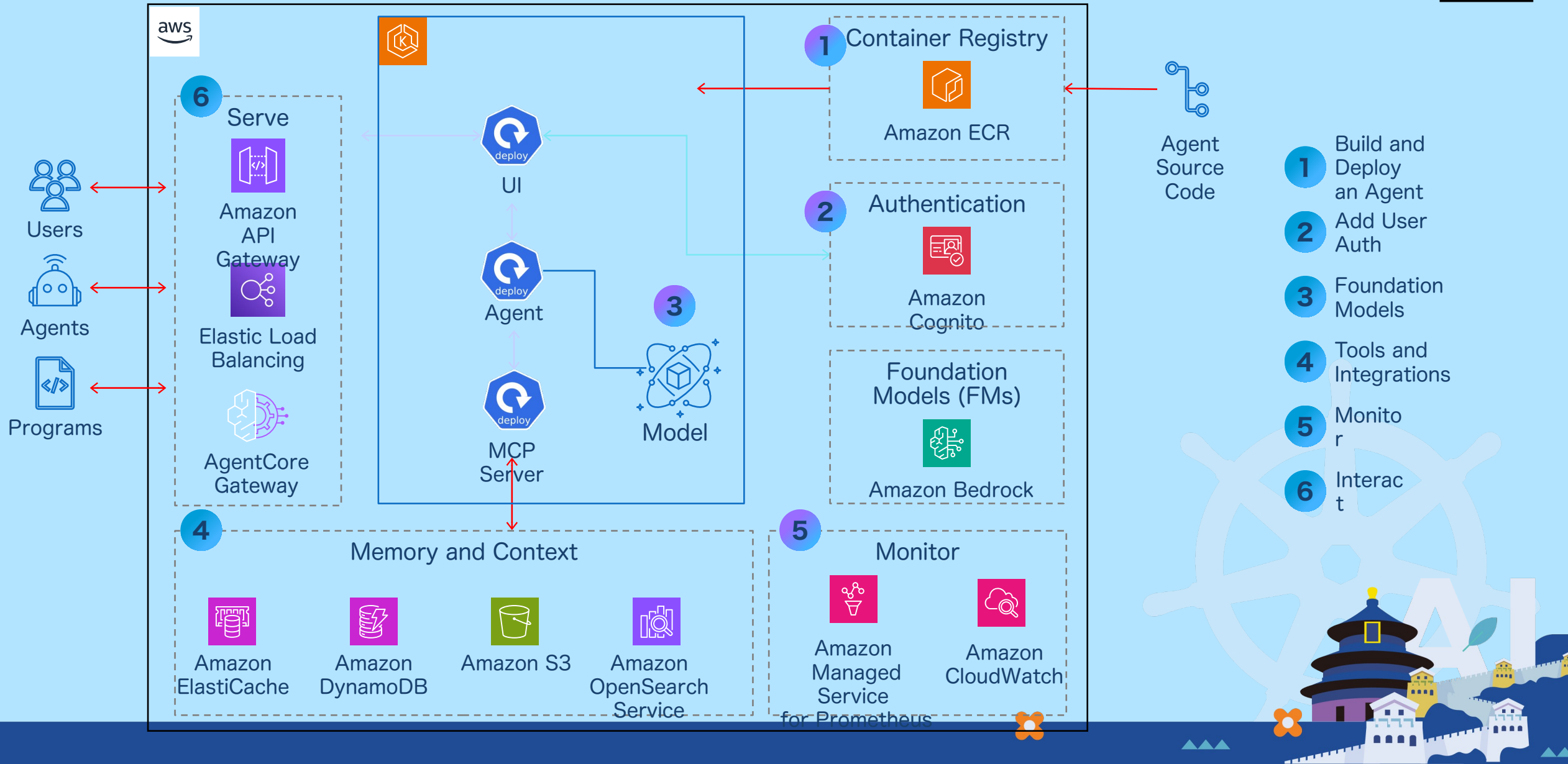
Detects accelerated instance driver failures and automatically reschedules workloads away from impaired GPUs



Standardizing on Kubernetes



E2E Agent and Model Deployment on EKS

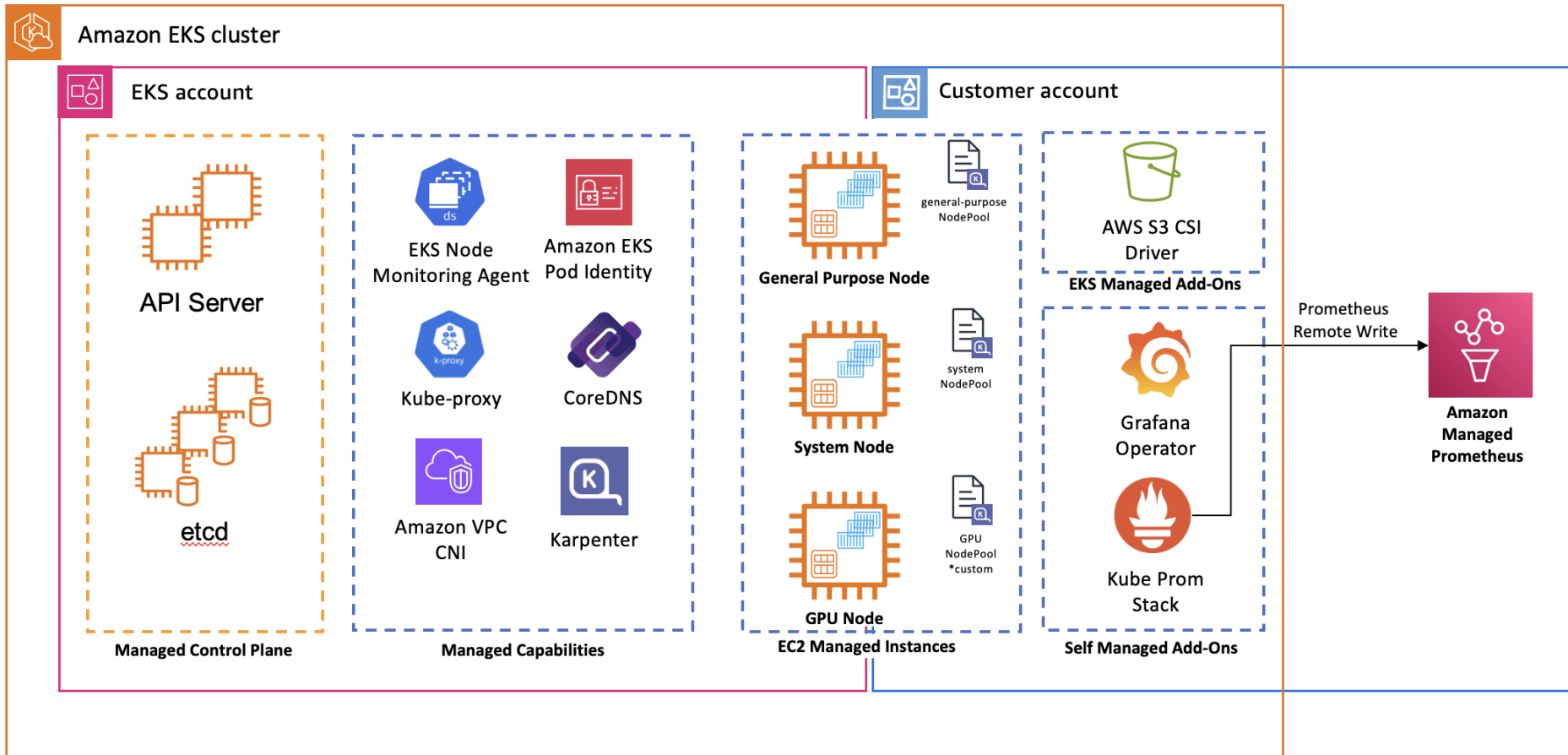


Part 02

Model Deployment with vLLM



Model Deployment Infrastructure Overview

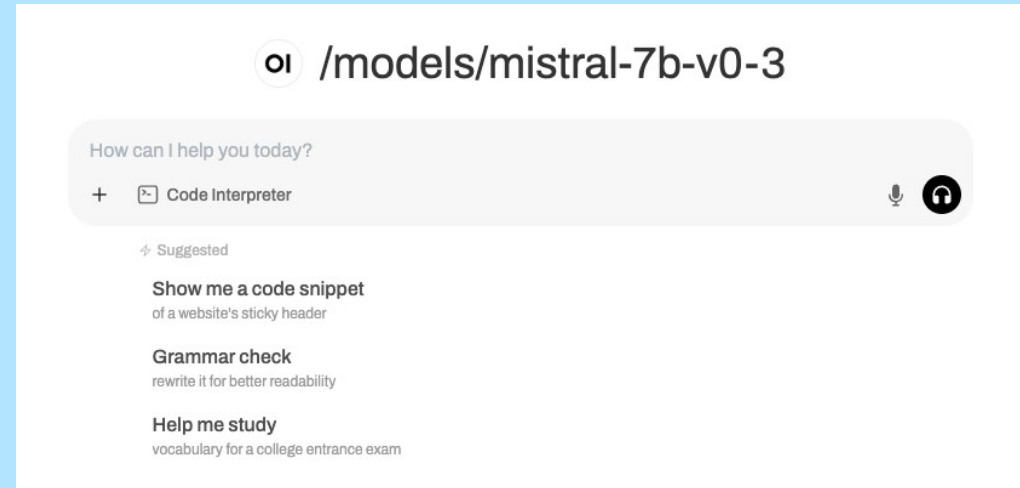


Demo Video
on RedNote

* EKS Auto Mode comes with both the node monitoring agent and node auto-repair enabled.

Model Deployment - Customizing the Manifest File

```
46 containers:
47   - name: vllm
48     image: 763104351884.dkr.ecr.us-east-1.amazonaws.com/vllm:0.10.2-gpu-py312-ec2
49     imagePullPolicy: IfNotPresent
50     args:
51       - '--port=8000'
52       - '--model=s3://genai-models-554456892689/mistral-7b-v0-3/'
53       - '--load-format=runai_streamer'
54       - '--model-loader-extra-config={"concurrency":16}'
55       - '--tokenizer_mode=auto'
56       - '--trust-remote-code'
57       - '--gpu_memory_utilization=0.90'
58       - '--max-model-len=2048'
59       - '--tensor-parallel-size=1'
60       - '--max-num-batched-tokens=8192'
61       - '--max-num-seqs=256'
62       - '--block-size=16'
63       - '--enforce-eager'
64       - '--swap-space=16'
65       - '--disable-custom-all-reduce'
66       - '--config-format=mistral'
67       - '--enable-auto-tool-choice'
68       - '--tool-call-parser=mistral'
69     env:
70       - name: CUDA_LAUNCH_BLOCKING
71         value: "1"
72       - name: PYTORCH_CUDA_ALLOC_CONF
73         value: "max_split_size_mb:512"
74       - name: VLLM_ATTENTION_BACKEND
75         value: "FLASHINFER"
76     ports:
77       - containerPort: 8000
78         protocol: TCP
79         name: http
80     resources:
81       requests:
82         cpu: 6
83         memory: 32Gi
84         nvidia.com/gpu: 1
85       limits:
86         cpu: 6
87         memory: 32Gi
88         nvidia.com/gpu: 1
```



Part 03

Observing LLM Inference Workloads



Install DCGM Exporter to Collect GPU Metrics for Prometheus

manifests > 200-inference > ! values.yaml

```
1  serviceMonitor:
2    enabled: true
3    additionalLabels:
4      release: kube-prometheus-stack # Important for prometheus operator discovery
5    interval: 30s
6    honorLabels: true
7
8  service:
9    enable: true
10   type: ClusterIP
11   port: 9400
12   annotations:
13     prometheus.io/scrape: "true"
14     prometheus.io/port: "9400"
15
16  nodeSelector:
17    karpenter.sh/nodepool: gpu
18
19  tolerations:
20    - key: "nvidia.com/gpu"
21      operator: "Exists"
22      effect: "NoSchedule"
23
24  # Pod annotations for Prometheus scraping
25  podAnnotations:
26    prometheus.io/scrape: "true"
27    prometheus.io/port: "9400"
28
29  # Pod labels
30  podLabels:
31    app.kubernetes.io/name: "dcgm-exporter"
32
33  # Resource limits to prevent OOM issues
34  resources:
35    limits:
36      cpu: 500m
37      memory: 512Mi
38    requests:
39      cpu: 100m
40      memory: 256Mi
41
42  # Extra environment variables
43  extraEnv:
44    - name: "DCGM_EXPORTER_LISTEN"
45      value: ":9400"
46    - name: "DCGM_EXPORTER_KUBERNETES"
47      value: "true"
```

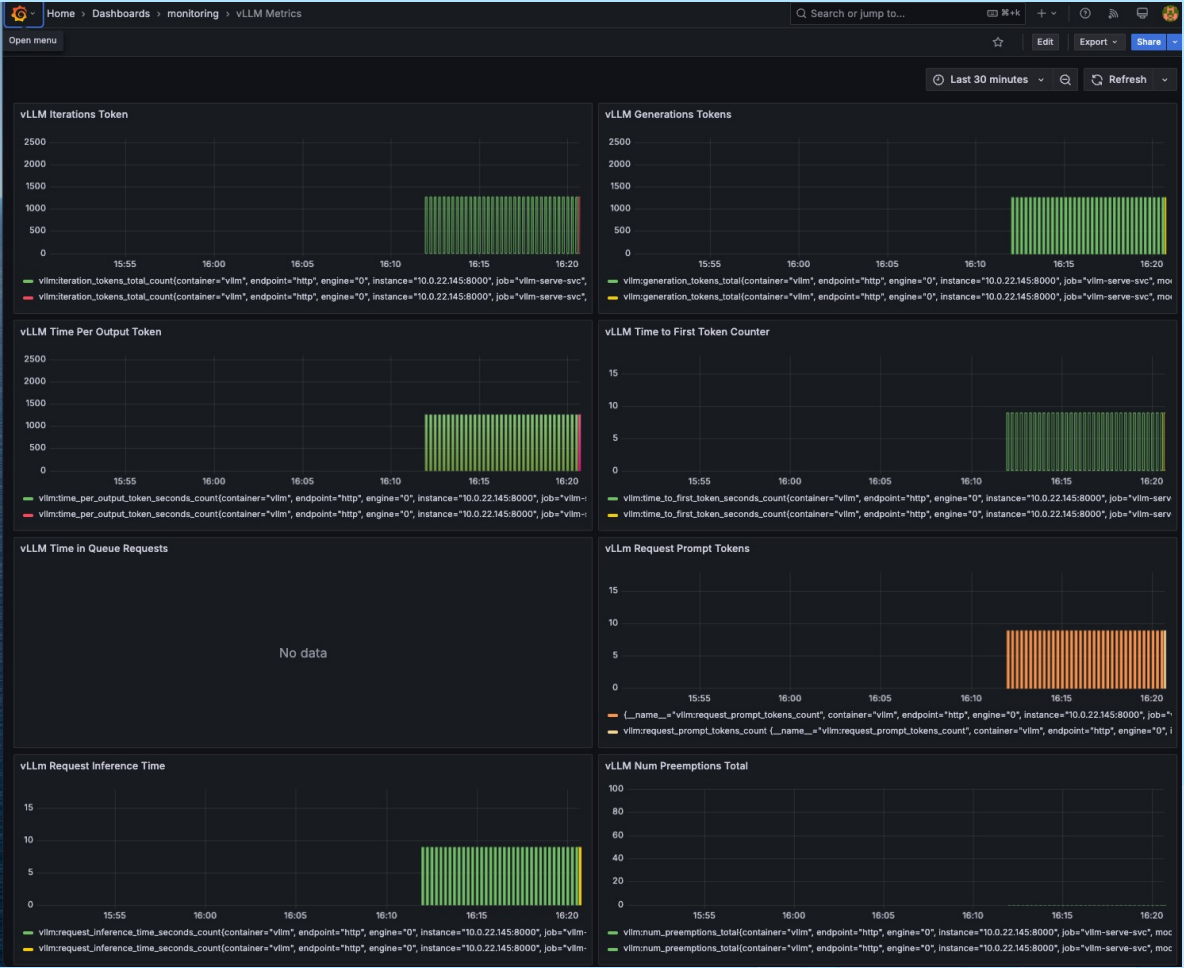
```
1 # Add NVIDIA Helm repository
2 helm repo add gpu-helm-charts https://nvidia.github.io/dcgm-exporter/helm-charts
3 helm repo update
```

```
1 # Install DCGM Exporter in the monitoring namespace
2 helm install dcgm-exporter gpu-helm-charts/dcgm-exporter \
3   -n monitoring \
4   -f manifests/200-inference/values.yaml
```

```
1 # Verify DCGM Exporter deployment
2 kubectl wait pods --for=jsonpath='{.status.phase}'=Running -l "app.kubernetes.io/name=dcgm-exporter" -n monitoring --timeout=300s
```



Observing Dashboard – DCGM & vLLM Metrics

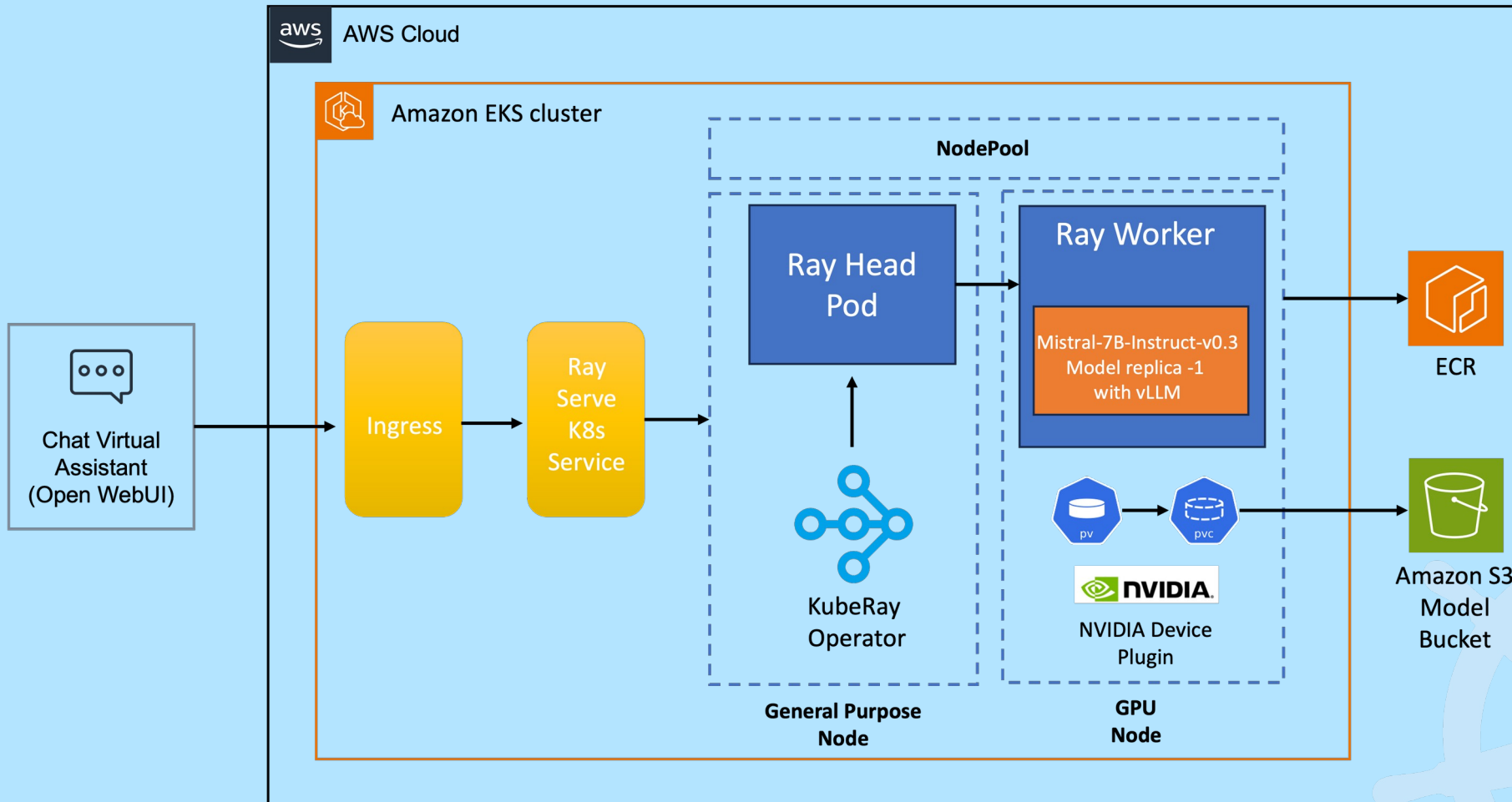


Part 04

Scaling LLM Inference with vLLM & Ray



Scaling LLM Inference using Ray & vLLM - Architecture



Demo Video
on RedNote



Scaling LLM Inference – vLLM Serving Script ConfigMap

```
manifests > 400-ray > ! vllm-s3-configmap.yml
1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: vllm-serve-script
5  data:
6    vllm_serve.py: |
7      import os
8      import re
9      from typing import Optional
10     from fastapi import FastAPI
11     from prometheus_client import make_asgi_app
12     from starlette.requests import Request
13     from starlette.responses import StreamingResponse, JSONResponse
14     from starlette.routing import Mount
15     from ray import serve
16     from vllm.engine.arg_utils import AsyncEngineArgs
17     from vllm.engine.async_llm_engine import AsyncLLMEngine
18     from vllm.entrypoints.openai.protocol import (
19         CompletionRequest,
20         CompletionResponse,
21         ChatCompletionRequest,
22         ChatCompletionResponse,
23         ErrorResponse,
24     )
25     from vllm.entrypoints.openai.serving_chat import OpenAIServingChat
26     from vllm.entrypoints.openai.serving_completion import OpenAIServingCompletion
27     from vllm.entrypoints.openai.serving_models import OpenAIServingModels, BaseModelPath
28     from vllm.config import ModelConfig
29     from vllm.logger import init_logger
30
31     logger = init_logger(__name__)
32     app = FastAPI()
33
34     @serve.deployment(
35         name="mistral-deployment",
```

Breaking Down the Full Python Code

- Application Setup
- vLLM Deployment Class
- API Endpoints
- Model Serving Infrastructure
- Configuration & Deployment



Scaling LLM Inference – KubeRay Operator & RayService Deployment

The **KubeRay Operator** offers significant advantages for production environments:

- **Zero-downtime upgrades** during model updates
- **High availability** through an external Redis for the Global Control Service
- **Automated cluster lifecycle management**
- **Native Kubernetes integration** via custom resources
- **Simplified scaling and monitoring**



Scaling LLM Inference – Ray Cluster Configuration

```
containers:
- name: ray-head
  image: public.ecr.aws/deep-learning-containers/vllm:0.10.2-gpu-py312-cu128-ubuntu22.04-rayserve-ec2-v1.0-soci
  imagePullPolicy: IfNotPresent
  lifecycle:
    preStop:
      exec:
        command: ["/bin/sh", "-c", "ray stop"]
  ports:
  - containerPort: 6379
    name: gcs
  - containerPort: 8265
    name: dashboard
  - containerPort: 10001
    name: client
  - containerPort: 8000
    name: serve
  - containerPort: 52365
    name: dashboard-agent
  volumeMounts:
  - mountPath: /tmp/ray
    name: ray-logs
  - name: vllm-script
    mountPath: /home/ray/python/vllm_serve.py
    subPath: vllm_serve.py
  - name: persistent-storage
    mountPath: /models
  resources:
    limits:
      cpu: 1
      memory: "6G"
    requests:
      cpu: 1
      memory: "6G"
```

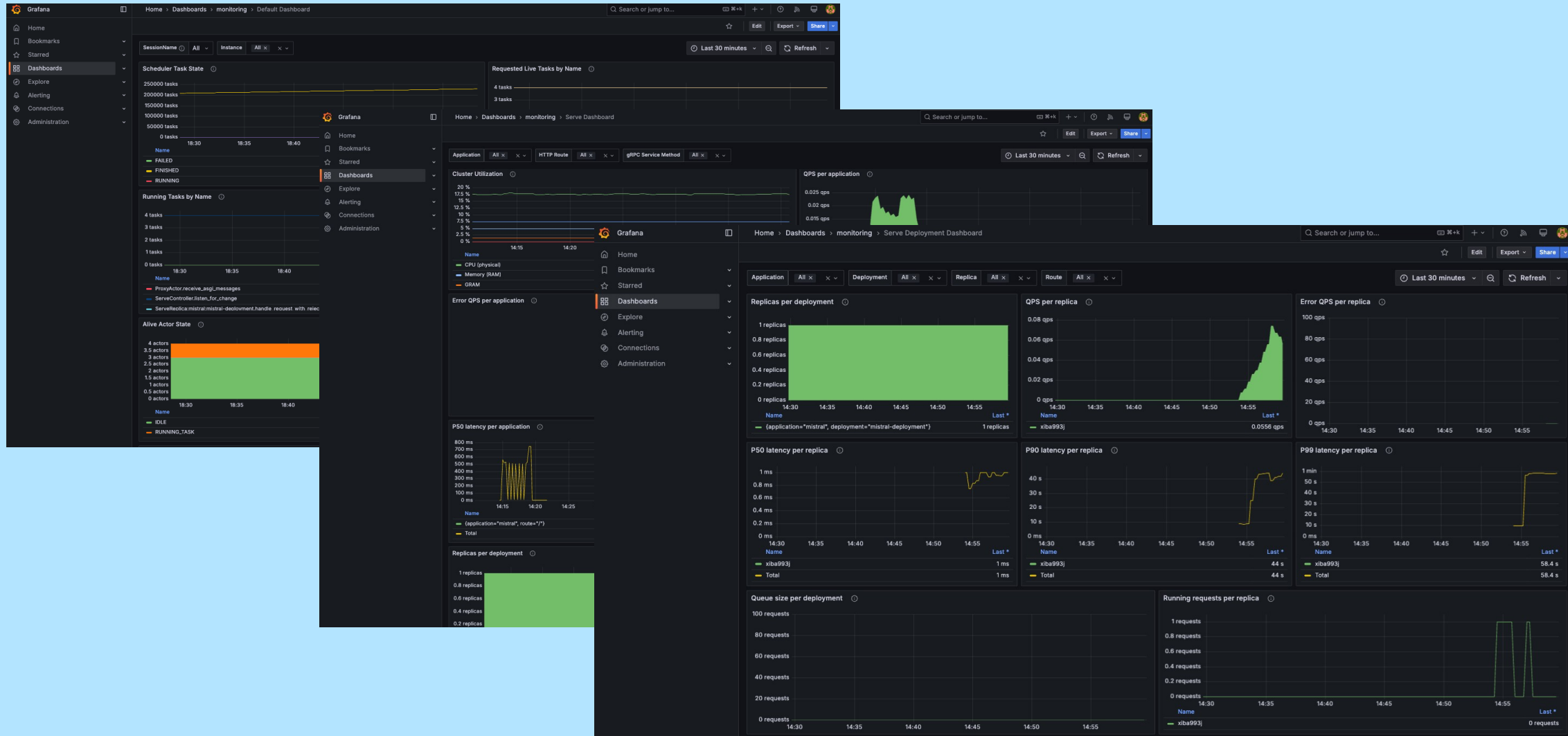
Head Node Configuration

```
containers:
- name: ray-worker
  image: public.ecr.aws/deep-learning-containers/vllm:0.10.2-gpu-py312-cu128-ubuntu22.04-rayserve-ec2-v1.0-soci
  imagePullPolicy: IfNotPresent
  lifecycle:
    preStop:
      exec:
        command: ["/bin/sh", "-c", "ray stop"]
  volumeMounts:
  - mountPath: /tmp/ray
    name: ray-logs
  - name: vllm-script
    mountPath: /home/ray/python/vllm_serve.py
    subPath: vllm_serve.py
  - name: persistent-storage
    mountPath: /models
  resources:
    limits:
      cpu: 5
      memory: "48Gi"
      nvidia.com/gpu: 1
    requests:
      cpu: 5
      memory: "48Gi"
      nvidia.com/gpu: 1
```

Worker Node Configuration



Scaling LLM Inference using vLLM & Ray – Monitoring



Large Language Model (LLM) Inference in Practice: Video Series

- Episode #1: Auto-provisioning GPU Nodes on EKS in 30 Seconds
- Episode #2: Deploying an LLM on EKS in 60 Seconds
- Episode #3: Real-time Monitoring of Inference Workloads
- Episode #4: Distributed Inference Deployment with vLLM and Ray



Available on RedNote
(Episodes 2 & 4)



Available on YouTube
(Episodes 1, 2, 3, & 4)



Hands-on Workshop Manual: A Step-by-Step Guide

- Workshop: Generative AI on Kubernetes (Amazon EKS)
 - <https://catalog.workshops.aws/genai-on-eks/en-US>



Thanks!



Haowen

HUANG



@haowenhuan
g

